arXiv:2412.04449v2 [cs.CV] 6 Aug 2025

p-MoD: Building Mixture-of-Depths MLLMs via Progressive Ratio Decay

Jun Zhang[1,*], Desen Meng[1,*], Zhengming Zhang [2], Zhenpeng Huang [1], Tao Wu [1], Limin Wang [1,3,✉]
[1]State Key Laboratory for Novel Software Technology, Nanjing University [2]China Mobile Research Institute [3]Shanghai AI Lab
https://github.com/MCG-NJU/p-MoD

Abstract

Despite the remarkable performance of multimodal large language models (MLLMs) across diverse tasks, the substantial training and inference costs impede their advancement. In this paper, we propose ***p-MoD****, an efficient MLLM architecture that significantly reduces training and inference costs while maintaining model performance. The majority of computation in MLLMs stems from the overwhelming volume of vision tokens processed by the transformer-based LLM. Accordingly, we leverage the* ***Mixture-of-Depths*** *(MoD) mechanism, where each LLM layer selects essential vision tokens to process while skipping redundant ones. However, integrating MoD into MLLMs is non-trivial. To address the challenges of training and inference stability as well as limited training data, we adapt the MoD module with two novel designs: tanh-gated weight normalization (****TanhNorm****) and symmetric token reweighting (****STRing****). Moreover, we observe that vision tokens exhibit higher redundancy in deeper layers and thus design a progressive ratio decay (****PRD****) strategy, which gradually reduces the token retention ratio layer by layer, employing a shifted cosine schedule. This crucial design fully unleashes the potential of MoD, significantly boosting the efficiency and performance of our models. Extensive experiments on two baseline models across 15 benchmarks show that our model matches or even surpasses the performance of corresponding baselines, while requiring only 55.6% TFLOPs and 53.7% KV cache storage during inference, and 77.7% GPU hours during training.*

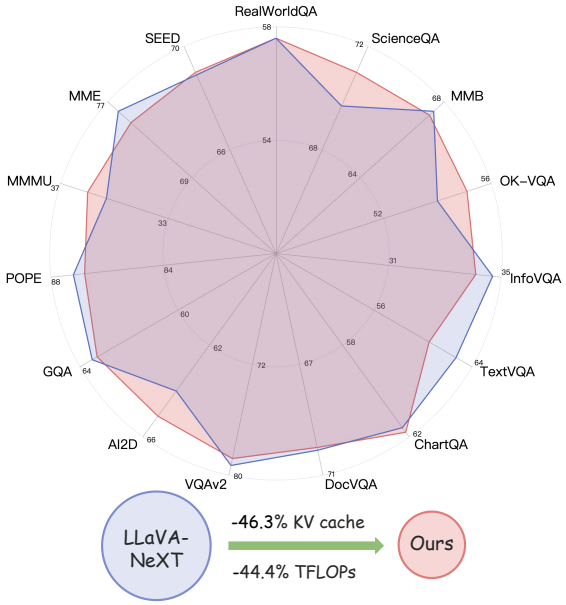


Figure 1. **Comparison with the LLaVA-NeXT [28] baseline.** Our model demonstrates comparable performance with the baseline model on 15 benchmarks across various domains, with 46.3% fewer KV cache storage and 44.4% fewer TFLOPs during inference.

1. Introduction

Recently, both academia and industry have witnessed the rapid development of multimdodal large language models (MLLMs) [1, 14, 24, 46, 47, 51, 55], which have demonstrated exceptional performance across various vision-language understanding tasks. Pioneering works in this field [8, 27, 29, 59] focused on processing single low-resolution image inputs. Subsequently, to meet the diverse demands of real-world applications, increasing efforts have broadened the forms of visual inputs that MLLMs can support, including multiple high-resolution images and videos [6, 24, 45, 47, 55].

Current state-of-the-art MLLMs handle high-resolution images either by dividing the original image into multiple slices [6, 28, 55] which are independently processed by the vision encoder, or by using a stronger vision encoder with improved positional encoding which can process any image at its native resolution [1, 31, 47]. Consequently, when processing multiple high-resolution images or videos, the number of vision tokens increases dramatically, proportional to the number of pixels and the number of images or video frames. The overwhelming volume of vision tokens processed by the transformer-based LLM results in explosion of computational costs, which severely hampers further de-

* Equal contribution. ✉ Corresponding author.

velopment and broader application of MLLMs. Designing more efficient MLLM architectures with minimal performance degradation has thus become an urgent challenge for the community.

Previous efforts primarily focus on compressing vision tokens **before** the LLM, either in the Vision Encoder or the multimodal projector [2, 6, 12, 38, 39, 42, 49, 50, 55, 56]. These approaches forces the LLM to handle heavily compressed vision information, overlooking the fact that the LLM, with its enormous model capacity, has the potential to compress the vision tokens by itself within the transformer layer.

In this paper, we focus on optimizing computation efficiency of MLLMs **within** the transformer layers of the LLM. We propose to build efficient MLLMs with Mixture-of-Depths (**MoD**) [41] mechanism, which selects the most important and informative vision tokens to be processed by each transformer layer, while skipping redundant ones to improve efficiency. However, integrating MoD mechanism into MLLMs is non-trivial and entails several significant challenges.

First, different from training an MoD-based LLM from scratch, integrating MoD mechanism into a pre-trained vanilla LLM during multimodal training poses a substantial risk of disrupting the language abilities of the original LLM. To address this, we design tanh-gated weight normalization (**TanhNorm**), which not only ensures proper initialization of the newly added MoD module, but also enhances training stability and performance. It also mitigates numerical stability issues during inference.

Second, MLLMs are trained on multimodal data [28] that are several orders of magnitude smaller in scale compared to the text data used for training MoD-based LLMs [41]. We enhance the MoD mechanism with symmetric token reweighting (**STRing**) module which fully leverage the language supervision signals during training, enabling MoD modules to learn to accurately assess token importance even with limited training data.

Thanks to these two enhancements, our upgraded MoD layers can be seamlessly applied to MLLMs. However, we argue that setting a **fixed** ratio of retained tokens across different MoD layers [41] is a suboptimal design choice under multimodal scenario, as the degree of redundancy in vision tokens should vary across layers. We conducted a series of exploratory experiments by adjusting the ratio of different layers in our MoD-based MLLM, which demonstrate that vision tokens exhibit **higher** redundancy in **deeper** layers. Accordingly, we propose a progressive ratio decay (**PRD**) strategy, which follows a shifted cosine schedule to gradually reduce the token retention ratio layer by layer. Trained with this strategy, our model significantly outperforms models that use a constant retention ratio across all layers under the same computation budget.

Building upon the above innovations, we present our model, ***p-MoD***, Mixture-of-Depth MLLMs equipped with our ***p***rogressive ratio decay strategy and upgraded MoD layers (*i.e.* ***p-MoD*** layers). Extensive experiments validate the effectiveness of our proposed model. As shown in Figure 1, across 15 benchmarks spanning various domains, our model matches or even outperforms the strong LLaVA-NeXT [28] baseline, with only 55.6% TFLOPs and 53.7% KV cache storage during inference, and 77.7% GPU hours during training.

2. Related Work

Building Efficient LLMs. Considerable efforts have been made to build efficient LLMs. One representative approach is the Mixture of Experts (**MoE**) mechanism [7, 9, 19, 22, 43, 60], where a router directs each token to different MLP experts of the transformer block. MoE models achieve faster training and inference speeds compared to dense models of the same size, but at the cost of lower performance. The more recently proposed Mixture of Depths (**MoD**) [41] module assigns weights to tokens and processes only a portion of highest-weighted tokens, skipping lower-weighted tokens to save computation. MoD-based LLMs demonstrate strong performance under limited compute budgets. In this paper, we propose to build more efficient MLLMs by upgrading MoD with several innovative improvements.

Building Efficient MLLMs. The main computational burden in MLLMs arises from the large number of vision tokens that the LLM must process. Previous works mainly focused on compressing vision tokens **before** they enter the LLM via convolution [6, 55], pooling [38, 50], query-based modules [2, 16] or token merging [3, 15, 25, 42, 49]. However, reducing vision-related computation **within** the LLM layers is less explored. LLaVolta [4] speeds up MLLM training by performing naive average pooling operation in intermediate LLM layers. FastV [5] and some subsequent works [13, 15, 17, 48, 57] improve MLLMs' efficiency by compressing vision tokens in intermediate transformer layers based on attention scores. However, these methods neglect the fact that tokens compressed in early layers might be crucial in deeper layers, potentially leading to performance degradation. In contrast, we utilize MoD for learnable **layerwise** vision token **selection**, improving model efficiency while avoiding potential performance loss caused by token dropping.

3. Method

In this section, we introduce our ***p-MoD*** model. As illustrated in Figure 2, our model consists of ***p-MoD*** layers which upgrades MoD architecture with tanh-gated weight

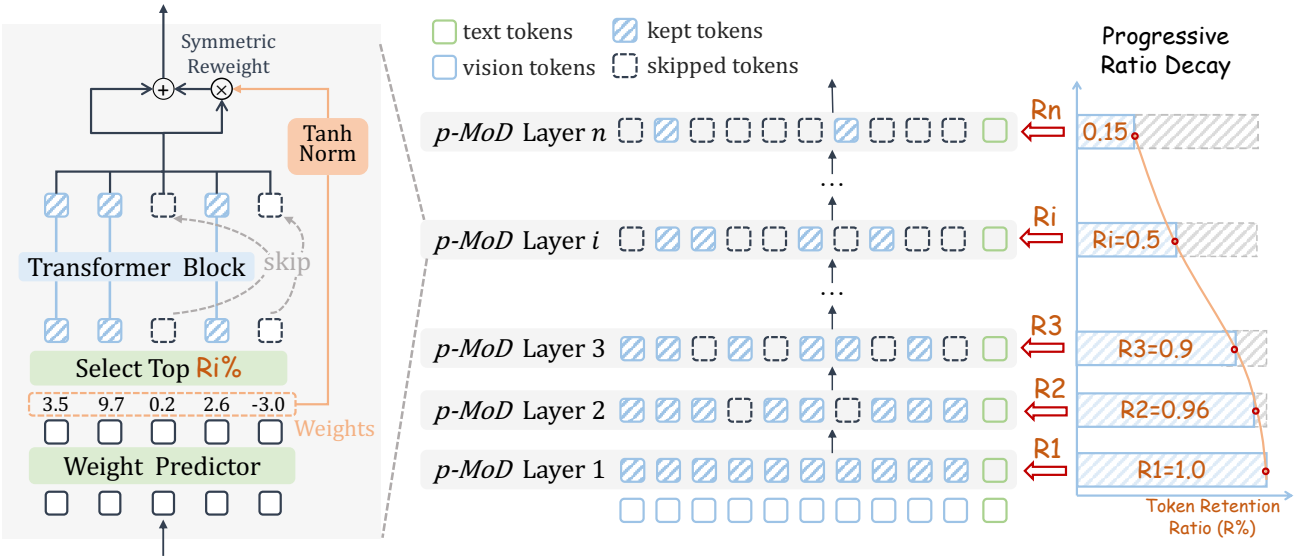


Figure 2. **Overview of *p-MoD***. *Middle*: Our efficient MLLM model which consists of ***p-MoD*** layers. Each layer independently selects important vision tokens to process while skipping redundant ones. The proportion of selected tokens for layer i is specified by the token retention ratio R_i. For simplicity, the vision encoder and connector are omitted. *Left*: Detailed architecture of ***p-MoD*** layers. Given the input tokens, the weight predictor first assigns weights to each token. The top $R_i\%$ of tokens with highest weights are selected and processed by the transformer layer, while the rest of the tokens are skipped. The weights are normalized by **TanhNorm** module, and then both the selected and skipped tokens are *symmetrically* scaled by their corresponding weights in our **STRing** module. *Right*: Our crucial design is the progressive ratio decay(**PRD**) strategy, which gradually reduces the token retention ratio R_i layer by layer, following a shifted cos schedule.

normalization(**TanhNorm**) and symmetric token reweighting(**STRing**). Our crucial design is the progressive ratio decay(**PRD**) strategy which controls the token retention ratio across different layers. In the following sections, we first revisit MoD briefly and then explain each component of our ***p-MoD*** model in detail.

3.1. Revisiting Mixture-of-Depths

The MoD layer consists of a weight predictor and a vanilla transformer layer. It assigns weights for each input token using the weight predictor and then selects the top $R\%$ tokens with the highest weights to be processed by the transformer layer. Formally, given an MoD layer M with a token retention ratio R, a transformer layer T, and a linear weight predictor, the input tokens $X \in \mathbb{R}^{n \times d}$ is first passed through the linear predictor to generate a set of weights:

$$w = \text{Linear}(X) \in \mathbb{R}^n, \tag{1}$$

with n denoting the sequence length and d denoting the embedding dimension.

Then, we compute the R-th percentile of the router weights, denoted as $P_R(w)$. Tokens with weights larger than $P_R(w)$ will be selected to be processed by the transformer layer T while other tokens are skipped in this layer, which guarantees only $n \times R\%$ tokens are processed. The calculation process of an MoD layer can be formulated as:

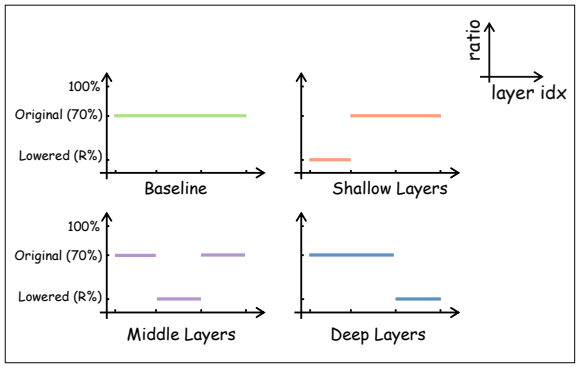
$$X'_i = \begin{cases} w_i T(X_i) + T(X_i), & \text{if} \quad w_i > P_R(w) \\ X_i, & \text{if} \quad w_i \leq P_R(w) \end{cases}. \tag{2}$$

Notably, after being processed by the transformer layer, the selected tokens are then scaled by their corresponding weights. In this way, the weight predictor is engaged into the gradient path, enabling its parameters to be updated by backpropagation. We term this process *token reweighting*.

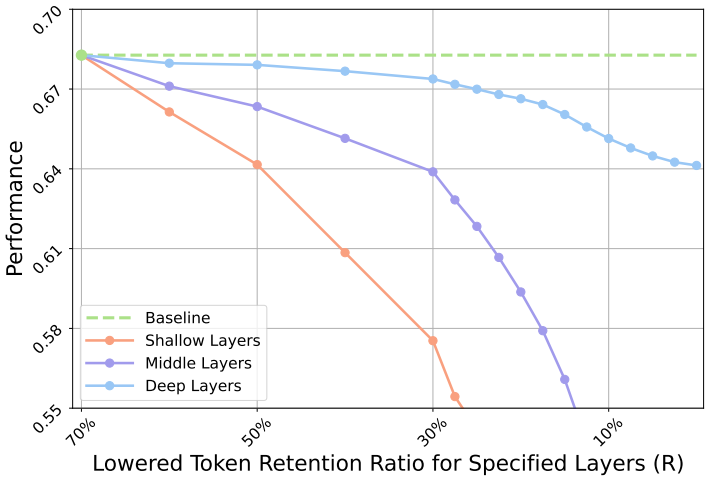
In the original MoD module designed for LLMs [41], X represents text tokens. In our multimodal scenario, X represents vision tokens. We only apply MoD on vision tokens as they occupy the main computation load and exhibit high redundancy in LLM layers.

3.2. Adapting Mixture-of-Depths Module

Different from training MoD-based LLMs from scratch, integrating MoD into MLLMs presents several challenges. The insertion of new MoD modules into pre-trained LLMs can lead to instability during training and inference. The relatively small amount of multimodal data may not be sufficient to train the MoD modules. In this section, we introduce two enhancements to the MoD module to address these challenges.



(a) **Illustration of our exploratory experiment setup.** Our baseline MoD model is trained with a token retention ratio of 70% for all layers. We divide the layers into three groups: shallow, middle, and deep. In each set of experiments, we decrease the token retention ratio (R%) in a specific group of layers.



(b) **Results of our exploratory experiments.** Performance drops more slowly when the token retention ratio of deeper layers is decreased, indicating that vision tokens are more *redundant* in *deeper* layers. This inspires us to *progressively reduce* the token retention ratio layer by layer.

Figure 3. **Exploratory experiments on vision token redundancy.**

3.2.1. Tanh-gated Weight Normalization

The original MoD mechanism [41] is designed for training MoD-based LLMs from scratch. In our multimodal training scenario, we need to insert new MoD modules into a pre-trained vanilla LLM. One common approach is alternate training [58], where only the newly inserted modules are trained in the first stage, and all modules are simultaneously trained in the second stage. However, this approach is not suitable for MoD, as the MoD modules scale the tokens by their weights during the token reweighting process, and the subsequent **frozen** LLM layers are unable to handle **scaled** tokens. Our experiments show consistent results that training the MoD modules while freezing the LLM layers fails to converge. Consequently, the only feasible method is to directly address the challenge: inserting the MoD modules into the LLM and training both of them simultaneously.

To address this challenge, we design tanh-gated weight normalization (**TanhNorm**), which employs a normalization function $f(w) = \alpha \tanh(w)$ to normalize the predicted weights before token reweighting. After applying **TanhNorm** to MoD, Equation 2 can be reformulated as:

$$X_i' = \begin{cases} \alpha \tanh(w_i) T(X_i) & \\ \qquad\qquad + T(X_i), & \text{if} \quad w_i > P_R(w) \\ X_i, & \text{if} \quad w_i \leq P_R(w) \end{cases} . \quad (3)$$

Here α is a hyper-parameter which controls the range of the normalized weights. Although simple, our **TanhNorm** ensures: (1) the normalized weight distribution is zero-centered, (2) the range(variance) of the weight distribution can be easily controlled by adjusting the gating factor α. These properties offer the following benefits:

- The normalized weights are closely around zero at the start of training, which ensures proper training initialization by keeping the LLM intact after inserting MoD modules.
- The zero-centered weight distribution reduced the risk that some tokens are repetitively scaled by positive(or negative) weights in all MoD layers. This ensures training stability and mitigates numerical stability issues during inference.

Both of the above benefits contribute to improved model performance. We validate the effectiveness of **TanhNorm** in Section 4.3.

3.2.2. Symmetric Token Reweighting

Compared to the text data used for training MoD-based LLMs, MLLMs are trained on multimodal data [28] that are several orders of magnitude smaller in scale. It is challenging to train a weight predictor (the only learnable part in the MoD module) to accurately and robustly assess the importance of vision tokens and assign corresponding weights.

As stated in Section 3.1, the gradient of the weight predictor module stems from the token reweighting process. The original MoD performs token reweighting only on selected tokens as in Equation 2. In this way, the language supervision signals only supervise the weight predictor to assign high weights to the selected tokens, while the process of predicting weights for the skipped tokens is not supervised (*i.e.* no gradient).

To fully leverage the limited training data, we enhance the MoD mechanism by symmetrically applying the token reweighting process to both selected and skipped tokens. With our symmetric token reweighting (**STRing**) module,

Model	Inference TFLOPs ↓	Inference KV cache ↓	Doc VQA	Chart QA	Text VQA	Info VQA	RW QA	G QA	OK VQA	PO PE	AI 2D	SE ED	AVG
LLaVA-v1.5	8.38	100%	28.1	18.2	46.0	25.8	55.6	61.9	53.4	85.9	55.2	66.2	49.6
+ ***p-MoD***	4.92(−41.3%)	53.7%	27.6	16.8	44.8	26.8	55.7	62.2	56.0	85.5	56.2	66.5	49.8
LLaVA-NeXT	39.46	100%	70.1	61.6	62.7	34.7	57.6	63.5	54.0	87.2	64.0	68.9	62.4
+ ***p-MoD***	21.94(−44.4%)	53.7%	70.0	61.8	60.5	34.1	57.6	63.3	55.1	86.8	65.1	69.0	62.3

Table 1. **Comparison with baseline models on 10 benchmarks.** Our models (marked in gray) achieves comparable or even better performance compared to baseline models, with only 55.6% TFLOPs and 53.7% KV cache storage during inference. All models are of 7B parameter scale.

Model	S QA	MM B	MM MU	VQA v2	MM E	AVG
LLaVA-v1.5	69.7	64.1	36.6	76.6	1506.8	64.5
+ ***p-MoD***	69.3	65.4	36.3	76.9	1482.8	64.4
LLaVA-NeXT	69.7	67.5	35.3	79.3	1519.3	65.6
+ ***p-MoD***	71.0	67.3	36.0	78.8	1495.5	65.6

Table 2. **Comparison with baseline models on more benchmarks.** Our ***p-MoD*** models (marked in gray) matches the performance of corresponding baseline models. SQA stands for ScienceQA-IMG. All models are 7B in size.

Model	Doc VQA	Chart QA	Text VQA	G QA	SE ED	MM B	AVG
vanilla MoD	61.1	54.0	55.8	61.8	66.9	63.1	60.4
+ **TanhNorm**	61.8	54.5	56.4	62.5	67.1	65.9	61.4
+ **STRing**	65.7	58.4	59.3	63.0	67.1	66.8	63.4
+ **PRD**	**70.0**	**61.8**	**60.5**	**63.3**	**69.0**	**67.3**	**65.3**

Table 3. **Ablation study on our propsed innovations.** Under fair comparison, all of our proposed modules (TanhNorm, STRing, and PRD) significantly improve model performance *without* any computational overhead, making indispensable contributions to the strong performance of our final model (marked in gray).

Equation 3 can be further modified as:

$$X'_i = \begin{cases} \alpha \tanh(w_i) T(X_i) & \\ \qquad\qquad + T(X_i), & \text{if} \quad w_i > P_R(w) \\ \alpha \tanh(w_i) X_i + X_i, & \text{if} \quad w_i \leq P_R(w) \end{cases} . \quad (4)$$

In this way, the MoD module can fully leverage the language supervision signals during training and learn to accurately assess token importance with limited training data.

3.3. Progressive Ratio Decay

After upgrading MoD with **TanhNorm** and **STRing** modules, we are able to successfully train MoD-based MLLMs. However, the performance-efficiency trade-off exhibited by the model is far from satisfactory. When the token retention ratio is set below 70%, the model's performance deteriorates sharply.

Original MoD-based LLMs [41] adopt a fixed token retention ratio across all MoD layers. This strategy assumes that the tokens have the same degree of redundancy in different layers. We argue that this assumption does not hold in the multimodal scenario. Under the effect of self-attention in every transformer layer, vision tokens gradually aggregate information from each other and text tokens gather relevant information from vision tokens. Therefore, vision tokens are expected to become increasingly redundant in deeper layers.

To validate our hypothesis, we conduct a series of exploratory experiments on our MoD-based MLLM trained with a token retention ratio of 70% across all layers. As shown in Figure 3a, we first divide the layers into several groups: shallow, middle, and deep layers. In each set of experiment, we gradually decrease the token retention ratio (R%) in a specific group of layers while keeping the other layers unchanged, and evaluate the model under this customized inference setting. The results in Figure 3b strongly support our hypothesis: performance drops more *slowly* when the token retention ratio of *deeper* layers is decreased. This suggests that vision tokens exhibit higher redundancy in deeper layers, which is consistent with observations in previous works [5, 57].

Accordingly, we design a progressive ratio decay (PRD) strategy. As shown on the right side of Figure 2, **PRD** gradually reduces the token retention ratio layer by layer, following a **shifted cosine schedule**. Suppose the model has L layers, the token retention ratio for the l-th layer is formulated as:

$$R_l = \frac{1}{2} \cos \frac{\pi l}{L} + \beta, \quad l = 1, 2, ..., L. \quad (5)$$

Here β is a **shift factor**, which can be used to flexibly control the overall computational cost of the model by vertically shifting the cosine decay curve.

Experiments in Section 4.3 demonstrates that our ***p-MoD*** model with **PRD** strategy significantly outperforms models that use a constant retention ratio under the same computation budget. It also outperforms other kinds of ratio schedulers.

Ratio Scheduler		Decay	Progressive	Doc VQA	Chart QA	Text VQA	G QA	SE ED	MM B	AVG
Constant		✗	✗	58.0	51.6	56.1	62.1	64.5	62.8	59.2
Interleaved		✗	✗	60.3	54.8	57.0	62.3	65.3	62.1	60.3
Stepped		✔	✗	67.2	61.2	59.4	63.5	68.4	66.1	64.3
PRD	Linear	✔	✔	69.3	60.4	**60.5**	**63.8**	68.0	66.7	64.8
	Cosine			**70.0**	**61.8**	**60.5**	63.3	**69.0**	**67.3**	**65.3**

Table 4. **Ablation on different token retention ratio schedules.** Our default model is marked in gray.

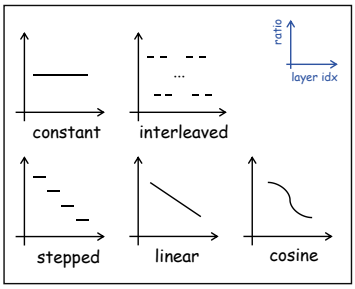


Figure 4. **Illustration of different ratio schedule functions in Table 4.**

Model	Token Compression Ratio ↓	Infer. TFLOPs	Doc VQA	Chart QA	Text VQA	Info VQA	RW QA	GQA	SQA	MM MU	PO PE	AI 2D	VQA v2	SE ED	AVG
LLaVA-NeXT	100%	39.46	70.1	61.6	62.7	34.7	57.6	63.5	69.7	35.3	87.2	64.0	79.3	68.9	62.9
+ MQT [16]	50.2%	19.86	49.9	44.0	53.5	29.8	53.5	62.3	69.1	36.1	85.8	64.0	77.2	65.8	57.6
+ LLaVolta [4]	53.1%	21.30	66.6	58.8	60.1	33.2	56.9	**63.7**	70.2	**36.7**	86.3	**65.3**	78.6	68.5	62.0
+ FastV [5]	53.1%	22.73	65.9	58.2	**62.2**	33.0	55.6	62.4	69.9	35.6	84.5	63.6	78.7	68.2	61.5
+ ***p-MoD***	53.7%	21.94	**70.0**	**61.8**	60.5	**34.1**	**57.6**	63.3	**71.0**	36.0	**86.8**	65.1	**78.8**	**69.0**	**62.8**

Table 5. **Comparison with other vision token compression methods.** ***p-MoD*** significantly outperforms other compression methods, achieving the best performance on most benchmarks and the best average performance. All models are of 7B parameter scale. Results on 13B models are demonstrated in Table 8.

4. Experiment

4.1. Setups

Models. To evaluate the effectiveness of ***p-MoD***, we select two representative open-source MLLMs: LLaVA-1.5 [27] and LLaVA-NeXT [28], as the baseline models in our experiments. LLaVA-1.5 resizes input images to the fixed resolution which aligns with the training setup of its CLIP [40] vision encoder, encoding an image into 576 tokens. LLaVA-NeXT divides high-resolution images into multiple slices which are independently processed by the vision encoder. This strategy enhances the visual perception capabilities, but also leads to a significantly larger number of vision tokens (up to 2880) and much higher computation costs.

Training Recipe. Our ***p-MoD*** models follow the two-stage training recipe of the baselines. In the pre-training stage, the MLP connector is trained on image caption data. In the fine-tuning stage, ***p-MoD*** modules are insert into the LLM and updated together with the connector and the LLM.

Benchmarks. We conduct comprehensive experiments across 15 benchmarks: VQAv2 [11], GQA [18], OK-VQA [34], AI2D [21] and ScienceQA-IMG [32] are traditional visual question answering benchmarks; DocVQA [36], TextVQA [44], ChartQA [35] and InfographicVQA [37] focus on fine-grained visual question answering; POPE [26] evaluates hallucination in MLLMs; SEED-Bench (Image) [23], RealWorldQA, MME [10], MMBench [30] and MMMU [53] are comprehensive benchmarks tailored for MLLMs. To ensure our results can be conveniently reproduced, we evaluate our model on all these benchmarks with the lmms-eval [54] evaluation framework.

Evaluation Metrics. In addition to performance on benchmarks, we present various metrics that reflect training and inference efficiency. We report TFLOPs which measures the inference computation complexity, and the KV cache storage which constitutes the main memory bottleneck during inference. Furthermore, we measure GPU hours consumed during training, along with inference latency.

4.2. Main Results

We integrate ***p-MoD*** with LLaVA-1.5 and LLaVA-NeXT baseline models and conduct comprehensive evaluation across 15 benchmarks, which measures comprehensive abilities of MLLMs across various aspects. The results are shown in Table 1 and Table 2. Both ***p-MoD***-LLaVA-v1.5 and ***p-MoD***-LLaVA-NeXT achieve comparable or even better performance across all benchmarks compared to their baseline models, with significant savings in inference TFLOPs and inference KV cache savings.

In Table 1, it is noteworthy that text-rich and graph-based benchmarks like DocVQA, ChartQA, TextVQA, and InfoVQA require fine-grained visual perception and reasoning abilities. Models can only given a correct answer when they successfully identify answer-related regions that occupy only small portions of the entire image. Remarkably, ***p-MoD***-LLaVA-NeXT achieves negligible performance drop on DocVQA and InfoVQA, and even gains 0.2% accuracy on ChartQA with substantial improvements

in time and memory efficiency.

The above results indicate that our approach substantially improves efficiency while maintaining performance.

4.3. Ablation Study

Effectiveness of propsed innovations. We conduct ablation in Table 3 to demonstrate the effectiveness of our proposed innovations. All experiments in the table are conducted under the *same* computational cost. TanhNorm, STRing and PRD *all* significantly enhance model performance, making indispensable contributions to the strong performance of our final ***p-MoD*** model.

Progressive Ratio Decay. Based on the exploratory experiments in Section 3.3, we conclude that vision tokens exhibit higher redundancy in deeper layers, and design the progressive ratio decay strategy with shifted cosine schedule.

To further validate this conclusion and the effectiveness of **PRD**, we experiment with four other schedule functions which control the token retention ratio for each LLM layer. As illustrated in Figure 4, the functions include: (1) a constant function that uses the same ratio for all layers, (2) an interleaved function which interleaves a vanilla transformer layer and an MoD layer with low ratio, which is adopted in the original MoD paper [41], (3) a stepped decay function, (4) a linear decay function. To ensure a fair comparison, we fix the average retention ratios across all layers at approximately 54%. Based on the results demonstrated in Table 4, we can draw the following conclusions:

- The non-decaying functions (*i.e.* constant and interleaved) significantly underperform the decaying functions. This result strongly supports our conclusion that vision tokens exhibit higher redundancy in deeper layers.
- Our progressively decayed **PRD** functions (*i.e.* cosine and linear) notably outperforms the discontinuous stepped decay function, and the cosine function achieves the best results. This proves the effectiveness of our shifted cosine schedule.

Tanh-gated Weight Normalization. To validate the effectiveness of **TanhNorm**, we compare different weight normalization methods in Table 6. First, we verify that using no weight normalization module will result in overflow during training, as shown in the first row (details are explained in appendix). Then, we validate the effectiveness of the two important properties of **TanhNorm** stated in Section 3.2.1: (1) the normalized weight distribution is zero-centered, (2) the range(variance) of the weight distribution can be easily controlled by adjusting the gating factor α.

For the first property, we design two experiments with the Softmax function, a standard normalization function that is *not* zero-centered. Specifically, we experiment with the vanilla softmax function $f(w) = \alpha \cdot Softmax(w), \alpha = 0.2$, and a shifted softmax function $f(w) = \alpha \cdot Softmax(w) + b, \alpha = 0.4, b = -0.2$ which has the same range as our **TanhNorm** function $f(w) = \alpha \tanh(w), \alpha = 0.2$.

Norm Type	Doc VQA	Chart QA	Text VQA	G QA	SE ED	MM B	AVG
-	OVERFLOW						-
Softmax	63.9	57.6	57.9	62.7	68.2	67.0	62.9
Shifted softmax	62.5	57.0	57.3	62.9	68.3	66.4	62.4
TanhNorm($\alpha = 1$)	OVERFLOW						-
TanhNorm($\alpha = 0.2$)	**70.0**	**61.8**	**60.5**	**63.3**	**69.0**	**67.3**	**65.3**

Table 6. **Ablation on tanh-gated weight normalization**. Our default model is marked in gray. The rows with no performance reported indicates that the corresponding experiment faces overflow issue during training or inference.

Comparing the last row of Table 6 with the second and third rows, we verify that **TanhNorm** significantly outperforms the vanilla softmax function and the shifted softmax function. It proves that being zero-centered is key to **TanhNorm**'s strong performance. The shifted softmax function, despite having the same range as **TanhNorm**, results in worse performance due to the lack of zero-centered property.

For the second property, we experiment with a naive choice of the gating factor α by setting it to 1, which can be interpreted as no gating is applied. As shown in the fourth row of Table 6, this experiment results in the overflow problem during inference. This validates the importance of controlling the gating factor α to guarantee training and inference stability.

4.4. Comparison with Related Works

In Table 5, we compare ***p-MoD*** with several strong token compression methods. MQT [16] employs a query transformer to compress vision tokens. LLaVolta [4] performs average pooling in intermediate LLM layers, achieving remarkable results despite its simplicity. FastV [5] drops vision tokens in intermediate LLM layers based on the attention score from text tokens. ***p-MoD*** significantly outperforms other compression methods, achieving the best performance on most benchmarks and the best average performance.

4.5. Efficiency-Performance Trade-off

Thanks to the shifted cosine schedule used by PRD in Section 3.3 to control the token retention ratio across all layers, we can change the shift factor β to control the overall computational cost of the model. In this way, we can train and deploy ***p-MoD*** models under different performance-efficiency trade-offs based on our actual needs.

To showcase the versatility of our approach, we conducted experiments with $\beta = 0.3, 0.4, 0.5$, respectively. The results are presented in Table 7. Our default ***p-MoD***

Model	Effiency				Benchmark						
	Training GPU hours ↓	Inference TFLOPs ↓	Inference Latency (ms) ↓	Inference KV cache storage ↓	Doc VQA	Chart QA	Text VQA	G QA	SE ED	MM B	AVG
LLaVA-NeXT-7B	560	39.46	519.1	100%	70.1	61.6	62.7	63.5	68.9	67.5	65.7
p-MoD-0.3	403 (−28.0%)	17.78 (−54.9%)	326.7 (−37.1%)	42.3%	65.5	57.8	58.1	63.1	68.6	67.0	63.4
p-MoD-0.4	415 (−25.9%)	19.56 (−50.4%)	347.1 (−33.1%)	47.5%	66.5	59.6	58.4	63.8	68.5	66.6	63.9
p-MoD-0.5	435 (−22.3%)	21.94 (−44.4%)	368.0 (−29.1%)	53.7%	70.0	61.8	60.5	63.3	69.0	67.3	65.3

Table 7. **The Efficiency-Performance trade-off experiments.** To showcase the versatility of our approach, we conducted experiments with $\beta = 0.3, 0.4, 0.5$, respectively. Inference latency is measured on TextVQA dataset.

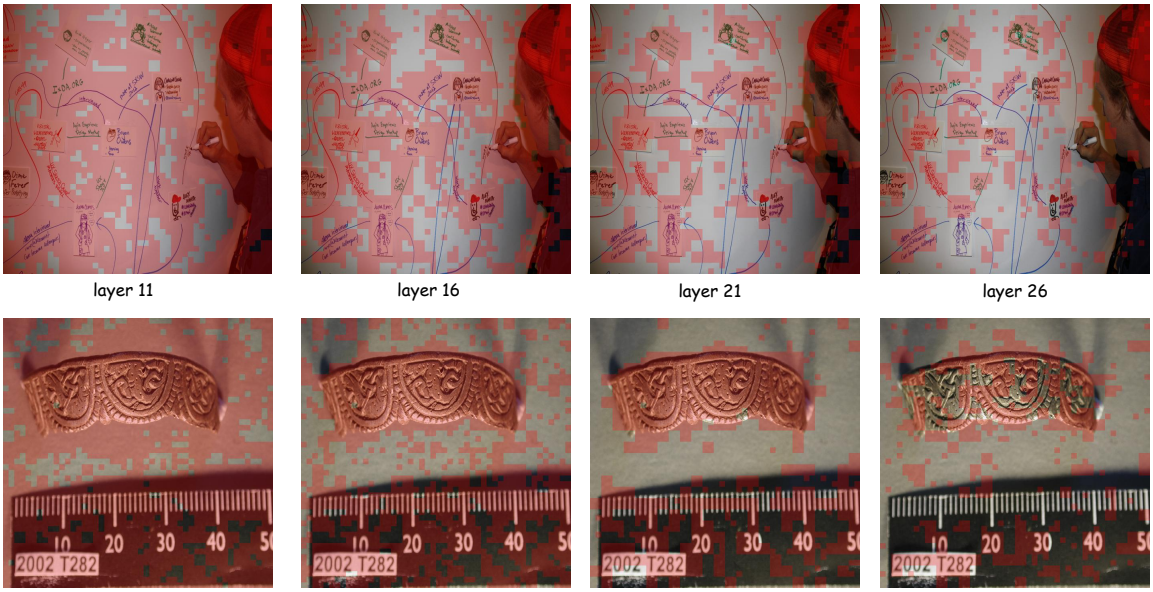


Figure 5. **Visualization of tokens selected by different *p-MoD* layers.** The selected tokens are colored in red. Please zoom in for a clearer view. As the number of selected tokens gradually decreases in deeper layers (due to **PRD** strategy), the model gradually concentrates on vision tokens corresponding to regions with **rich semantic information**.

model with $\beta = 0.5$ reduced training GPU hours by 22.3%, inference TFLOPs by 44.4%, inference latency by 29.1%, and KV cache storage by 46.3%. Using a lower β can further improve efficiency, with an acceptable performance drop.

4.6. Visualization: Which tokens are selected?

In Figure 5, we visualize the tokens selected by ***p-MoD***-LLaVA-NeXT at different layers. Guided by the **PRD** strategy, the number of tokens processed by the MoD layers gradually decreases layer by layer. It can be observed that the selected vision tokens progressively concentrate on key regions in the image with **rich semantic information**. For the upper row of images in Figure 5, the selected tokens gradually converge to those corresponding to text, drawings, and the person on the right. Tokens corresponding to the white background are skipped in deep layers. In the lower row of images in Figure 5, the selected tokens gradually converge towards the artifact and the measurement markings on the ruler. These results indicate that our model effectively compresses visual information by selecting the informative tokens, enabling it to maintain performance while significantly reducing training and inference costs.

5. Conclusion and Future Work

In this paper, we explore building efficient MLLMs by adapting the Mixture-of-Depths mechanism. Our proposed model, termed ***p-MoD***, features three key designs: tanh-gated weight normalization, symmetric token reweighting module, and the progressive ratio decay strategy. Our model achieves comparable or even superior results to the baseline models on a diverse set of 15 benchmarks, with substantial improvements on training and inference efficiency. We hope ***p-MoD*** can serve as a strong and efficient baseline for future research on developing efficient MLLMs.

Acknowledgements. This work is supported by the National Key R&D Program of China (No. 2022ZD0160900), Jiangsu Frontier Technology Research and Development Program (No. BF2024076), the Collaborative Innovation Center of Novel Software Technology and Industrialization, and Nanjing University-China Mobile Communications Group Co., Ltd. Joint Institute.

References

[1] Pravesh Agrawal, Szymon Antoniak, Emma Bou Hanna, Devendra Chaplot, Jessica Chudnovsky, Saurabh Garg, Theophile Gervet, Soham Ghosh, Amélie Héliou, Paul Jacob, et al. Pixtral 12b. *arXiv preprint arXiv:2410.07073*, 2024. 1

[2] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A frontier large vision-language model with versatile abilities. *arXiv preprint arXiv:2308.12966*, 2023. 2

[3] Wenhao Chai, Enxin Song, Yilun Du, Chenlin Meng, Vashisht Madhavan, Omer Bar-Tal, Jenq-Neng Hwang, Saining Xie, and Christopher D Manning. Auroracap: Efficient, performant video detailed captioning and a new benchmark. *arXiv preprint arXiv:2410.03051*, 2024. 2

[4] Jieneng Chen, Luoxin Ye, Ju He, Zhao-Yang Wang, Daniel Khashabi, and Alan Yuille. Llavolta: Efficient multi-modal models via stage-wise visual context compression. *arXiv preprint arXiv:2406.20092*, 2024. 2, 6, 7

[5] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*, pages 19–35. Springer, 2025. 2, 5, 6, 7

[6] Zhe Chen, Weiyun Wang, Hao Tian, Shenglong Ye, Zhangwei Gao, Erfei Cui, Wenwen Tong, Kongzhi Hu, Jiapeng Luo, Zheng Ma, et al. How far are we to gpt-4v? closing the gap to commercial multimodal models with open-source suites. *arXiv preprint arXiv:2404.16821*, 2024. 1, 2

[7] Damai Dai, Chengqi Deng, Chenggang Zhao, RX Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y Wu, et al. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. *arXiv preprint arXiv:2401.06066*, 2024. 2

[8] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. Instructblip: Towards general-purpose vision-language models with instruction tuning, 2023. 1

[9] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. 2

[10] Chaoyou Fu, Peixian Chen, Yunhang Shen, Yulei Qin, Mengdan Zhang, Xu Lin, Jinrui Yang, Xiawu Zheng, Ke Li, Xing Sun, Yunsheng Wu, and Rongrong Ji. Mme: A comprehensive evaluation benchmark for multimodal large language models. *arXiv preprint arXiv:2306.13394*, 2024. 6

[11] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the v in vqa matter: Elevating the role of image understanding in visual question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6904–6913, 2017. 6

[12] Kai Han, Jianyuan Guo, Yehui Tang, Wei He, Enhua Wu, and Yunhe Wang. Free video-llm: Prompt-guided visual perception for efficient training-free video llms. *arXiv preprint arXiv:2410.10441*, 2024. 2, 12

[13] Yefei He, Feng Chen, Jing Liu, Wenqi Shao, Hong Zhou, Kaipeng Zhang, and Bohan Zhuang. Zipvl: Efficient large vision-language models with dynamic token sparsification and kv cache compression. *arXiv preprint arXiv:2410.08584*, 2024. 2

[14] Wenyi Hong, Weihan Wang, Ming Ding, Wenmeng Yu, Qingsong Lv, Yan Wang, Yean Cheng, Shiyu Huang, Junhui Ji, Zhao Xue, et al. Cogvlm2: Visual language models for image and video understanding. *arXiv preprint arXiv:2408.16500*, 2024. 1

[15] Lianyu Hu, Fanhua Shang, Liang Wan, and Wei Feng. illava: An image is worth fewer than 1/3 input tokens in large multimodal models. *arXiv preprint arXiv:2412.06263*, 2024. 2, 12

[16] Wenbo Hu, Zi-Yi Dou, Liunian Harold Li, Amita Kamath, Nanyun Peng, and Kai-Wei Chang. Matryoshka query transformer for large vision-language models. *arXiv preprint arXiv:2405.19315*, 2024. 2, 6, 7

[17] Mingxin Huang, Yuliang Liu, Dingkang Liang, Lianwen Jin, and Xiang Bai. Mini-monkey: Alleviating the semantic sawtooth effect for lightweight mllms via complementary image pyramid. *arXiv preprint arXiv:2408.02034*, 2024. 2

[18] Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6700–6709, 2019. 6

[19] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024. 2

[20] Sahar Kazemzadeh, Vicente Ordonez, Mark Matten, and Tamara Berg. Referitgame: Referring to objects in photographs of natural scenes. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, pages 787–798, 2014. 12

[21] Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14*, pages 235–251. Springer, 2016. 6

[22] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020. 2

[23] Bohao Li, Rui Wang, Guangzhi Wang, Yuying Ge, Yixiao Ge, and Ying Shan. Seed-bench: Benchmarking multimodal llms with generative comprehension. *arXiv preprint arXiv:2307.16125*, 2023. 6

[24] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024. 1

[25] Xinhao Li, Yi Wang, Jiashuo Yu, Xiangyu Zeng, Yuhan Zhu, Haian Huang, Jianfei Gao, Kunchang Li, Yinan He, Chenting Wang, et al. Videochat-flash: Hierarchical compression for long-context video modeling. *arXiv preprint arXiv:2501.00574*, 2024. 2

[26] Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. *arXiv preprint arXiv:2305.10355*, 2023. 6

[27] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning, 2023. 1, 6, 14

[28] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, 2024. 1, 2, 4, 6, 14

[29] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36, 2024. 1, 14

[30] Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, et al. Mmbench: Is your multi-modal model an all-around player? In *European Conference on Computer Vision*, pages 216–233. Springer, 2025. 6

[31] Zuyan Liu, Yuhao Dong, Ziwei Liu, Winston Hu, Jiwen Lu, and Yongming Rao. Oryx mllm: On-demand spatial-temporal understanding at arbitrary resolution. *arXiv preprint arXiv:2409.12961*, 2024. 1

[32] Pan Lu, Swaroop Mishra, Tanglin Xia, Liang Qiu, Kai-Wei Chang, Song-Chun Zhu, Oyvind Tafjord, Peter Clark, and Ashwin Kalyan. Learn to explain: Multimodal reasoning via thought chains for science question answering. *Advances in Neural Information Processing Systems*, 35:2507–2521, 2022. 6

[33] Yaxin Luo, Gen Luo, Jiayi Ji, Yiyi Zhou, Xiaoshuai Sun, Zhiqiang Shen, and Rongrong Ji. γ-mod: Exploring mixture-of-depth adaptation for multimodal large language models. *arXiv preprint arXiv:2410.13859*, 2024. 12

[34] Kenneth Marino, Mohammad Rastegari, Ali Farhadi, and Roozbeh Mottaghi. Ok-vqa: A visual question answering benchmark requiring external knowledge. In *Proceedings of the IEEE/cvf conference on computer vision and pattern recognition*, pages 3195–3204, 2019. 6

[35] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. *arXiv preprint arXiv:2203.10244*, 2022. 6

[36] Minesh Mathew, Dimosthenis Karatzas, and CV Jawahar. Docvqa: A dataset for vqa on document images. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 2200–2209, 2021. 6

[37] Minesh Mathew, Viraj Bagal, Rubèn Tito, Dimosthenis Karatzas, Ernest Valveny, and CV Jawahar. Infographicvqa. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1697–1706, 2022. 6

[38] Brandon McKinzie, Zhe Gan, Jean-Philippe Fauconnier, Sam Dodge, Bowen Zhang, Philipp Dufter, Dhruti Shah, Xianzhi Du, Futang Peng, Floris Weers, et al. Mm1: Methods, analysis & insights from multimodal llm pre-training. *arXiv preprint arXiv:2403.09611*, 2024. 2

[39] Yasmine Omri, Parth Shroff, and Thierry Tambe. Token sequence compression for efficient multimodal computing. *arXiv preprint arXiv:2504.17892*, 2025. 2

[40] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021. 6

[41] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024. 2, 3, 4, 5, 7

[42] Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. Llava-prumerge: Adaptive token reduction for efficient large multimodal models. *arXiv preprint arXiv:2403.15388*, 2024. 2, 12

[43] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017. 2

[44] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards vqa models that can read. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8317–8326, 2019. 6

[45] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023. 1

[46] Peter Tong, Ellis Brown, Penghao Wu, Sanghyun Woo, Adithya Jairam Vedagiri IYER, Sai Charitha Akula, Shusheng Yang, Jihan Yang, Manoj Middepogu, Ziteng Wang, et al. Cambrian-1: A fully open, vision-centric exploration of multimodal llms. *Advances in Neural Information Processing Systems*, 37:87310–87356, 2024. 1

[47] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 1

[48] Long Xing, Qidong Huang, Xiaoyi Dong, Jiajie Lu, Pan Zhang, Yuhang Zang, Yuhang Cao, Conghui He, Jiaqi Wang, Feng Wu, et al. Pyramiddrop: Accelerating your large

vision-language models via pyramid visual redundancy reduction. *arXiv preprint arXiv:2410.17247*, 2024. 2, 12

[49] Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. Visionzip: Longer is better but not necessary in vision language models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 19792–19802, 2025. 2, 12

[50] Linli Yao, Lei Li, Shuhuai Ren, Lean Wang, Yuanxin Liu, Xu Sun, and Lu Hou. Deco: Decoupling token compression from semantic abstraction in multimodal large language models. *arXiv preprint arXiv:2405.20985*, 2024. 2

[51] Jiabo Ye, Haiyang Xu, Haowei Liu, Anwen Hu, Ming Yan, Qi Qian, Ji Zhang, Fei Huang, and Jingren Zhou. mplug-owl3: Towards long image-sequence understanding in multi-modal large language models. *arXiv preprint arXiv:2408.04840*, 2024. 1

[52] Licheng Yu, Patrick Poirson, Shan Yang, Alexander C Berg, and Tamara L Berg. Modeling context in referring expressions. In *European conference on computer vision*, pages 69–85. Springer, 2016. 12

[53] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruoqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9556–9567, 2024. 6

[54] Kaichen Zhang, Bo Li, Peiyuan Zhang, Fanyi Pu, Joshua Adrian Cahyono, Kairui Hu, Shuai Liu, Yuanhan Zhang, Jingkang Yang, Chunyuan Li, et al. Lmms-eval: Reality check on the evaluation of large multimodal models. *arXiv preprint arXiv:2407.12772*, 2024. 6

[55] Pan Zhang, Xiaoyi Dong, Yuhang Zang, Yuhang Cao, Rui Qian, Lin Chen, Qipeng Guo, Haodong Duan, Bin Wang, Linke Ouyang, et al. Internlm-xcomposer-2.5: A versatile large vision language model supporting long-contextual input and output. *arXiv preprint arXiv:2407.03320*, 2024. 1, 2

[56] Qizhe Zhang, Aosong Cheng, Ming Lu, Zhiyong Zhuo, Minqi Wang, Jiajun Cao, Shaobo Guo, Qi She, and Shanghang Zhang. [cls] attention is all you need for training-free visual token pruning: Make vlm inference faster. *arXiv e-prints*, pages arXiv–2412, 2024. 2, 12

[57] Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, et al. Sparsevlm: Visual token sparsification for efficient vision-language model inference. *arXiv preprint arXiv:2410.04417*, 2024. 2, 5, 12

[58] Yi-Fan Zhang, Qingsong Wen, Chaoyou Fu, Xue Wang, Zhang Zhang, Liang Wang, and Rong Jin. Beyond llava-hd: Diving into high-resolution large multimodal models. *arXiv preprint arXiv:2406.08487*, 2024. 4

[59] Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. Minigpt-4: Enhancing vision-language understanding with advanced large language models. *arXiv preprint arXiv:2304.10592*, 2023. 1

[60] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. St-moe: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022. 2

p-MoD: Building Mixture-of-Depths MLLMs via Progressive Ratio Decay

Supplementary Material

This supplementary material includes the following sections:

- In Section A, we provide more experiment results.
- In Section B, we show visualization results of ***p-MoD*** token selection decisions across different layers.
- In Section C, we make some further discussions about our work.
- In Section D, we provide more implementation details.

A. More Experiments

A.1. Experiments on Larger Model Size

In Table 8, we compare our method against other token compression methods on LLaVA-NeXT-13B baseline model. The results demonstrate that our method significantly outperforms other methods on larger model size, validating the scalability of our approach.

A.2. Experiments on Visual Grounding

In Table 9, we compare ***p-MoD*** with other token compression methods on visual grounding benchmarks RefCOCO, RefCOCO+ and RefCOCOg [20, 52]. Our method significantly outperforms other methods, but it still exhibits some performance drop compared to the baseline model. This suggests that current token pruning methods still exhibit limitations on visual grounding, which might be one of the major challenges that future works in this field should aim to address.

A.3. Compairson with More Relate Works

In addition to comparing ***p-MoD*** against three strong token compression methods in Table 5, we provide comparison with more methods in Table 10. Under the same token compression ratio, ***p-MoD*** demonstrates the best overall performance on our comprehensive evaluation suite covering 15 benchmarks.

A.4. Comparison with γ-MoD

Concurrent to our work, γ-MoD [33] also propose to integrate Mixture-of-Depths mechanism into MLLMs. In this section, we first analyze the difference between our approach and theirs. Then we conduct experiments to show that our ***p-MoD*** approach outperforms γ-MoD.

The core design of γ-MoD is computing attention map (ARank) on some samples to identify which layers MoD can be applied to. In contrast, ***p-MoD*** can be effectively applied to **every** layer thanks to our TanhNorm and STRing modules. Furthermore, we propose **PRD** strategy to progressively reduce the token retention ratio layer by layer, which significantly boosts performance and efficiency.

In Table 11, we compare our method with γ-MoD on LLaVA-v1.5-7B baseline. γ-MoD utilizes higher token retention ratio than ***p-MoD*** (60+% vs 53%), but ***p-MoD*** still outperforms it by a large margin. Note that we are unable to compare both methods on LLaVA-NeXT, as γ-MoD's code implementation does not support LLaVA-NeXT.

Method	DocVQA	ChartQA	SEED	GQA	AVG over **15** Tasks*
LLaVA-NeXT-13B	73.5	67.2	71.4	65.2	66.5
+ MQT	58.6	54.6	68.8	64.2	63.0
+ FastV	70.2	64.7	70.8	64.7	65.7
+ LLaVolta	70.0	61.7	70.5	64.7	65.0
+ ***p-MoD***	**72.3**	**66.2**	**71.6**	**65.0**	**66.0**

Table 8. **Experiments on LLaVA-NeXT-13B**. Our method significantly outperforms other methods on larger baseline model, validating the scalability of our approach. *Average is computed on all **15** benchmarks used in Table 1 and 2.

Method	RefCOCO Val	RefCOCO+ Val	RefCOCOg Val
LLaVA-NeXT-7B	82.67	73.73	79.41
+MQT	68.77	59.07	63.52
+LLaVolta	79.96	70.89	76.79
+FastV	73.83	64.90	69.78
+***p-MoD***	**80.23**	**70.94**	**76.96**

Table 9. **Experiments on Visual Grounding**. Our method significantly outperforms other methods on visual grounding benchmarks RefCOCO, RefCOCO+ and RefCOCOg. We report ACC@0.5 metric on the validation sets of these benchmarks.

Model	DocVQA	ChartQA	SEED	GQA	AVG over **15** tasks*
LLaVA-NeXT-7B	70.1	61.6	68.9	63.5	63.5
+ SparseVLM [57]	67.2	52.8	68.1	62.6	62.3
+ VisionZip [49]	65.5	50.3	67.4	61.1	61.2
+ PyramidDrop [48]	66.5	53.3	67.5	61.9	61.9
+ FasterVLM [56]	65.9	47.7	68.0	61.4	61.4
+ FreeVideoLLM [12]	55.5	36.0	68.9	59.3	57.6
+ iLLaVA [15]	64.5	56.6	65.5	61.5	59.3
+ LLaVA-PruMerge [42]	58.0	44.6	67.4	61.2	60.0
+ ***p-MoD***	**70.0**	**61.8**	**69.0**	**63.3**	**63.4**

Table 10. **Comparison with more vision token compression methods.** In addition to the comparison in Table 5, we make a fair comparison between ***p-MoD*** and more vision token compression methods by controlling the average token retention ratio. Our methods achieves the best overall performance across 15 benchmarks. *Average is computed on all **15** benchmarks used in Table 1 and 2.

Model	Keep Ratio↓	Doc VQA	Info VQA	Chart QA	Text VQA	RW QA	SE ED	PO PE	MM MU	AI 2D	VQA v2	OK VQA	MM E	G QA	S QA	MM B	AVG
LLaVA-v1.5	100%	28.1	25.8	18.2	46.0	55.6	66.2	85.9	36.6	55.2	76.6	53.4	1506.8	61.9	69.7	64.1	54.6
+ γ-MoD	$>60\%$	20.4	21.5	**18.1**	**47.5**	53.7	66.3	**86.6**	35.0	55.1	**77.1**	51.3	1,377.0	62.1	67.2	59.9	52.7
+ ***p-MoD***	**53.7%**	**27.6**	**26.8**	16.8	44.8	**55.7**	**66.5**	85.5	**36.3**	**56.2**	76.9	**56.0**	**1482.8**	**62.2**	**69.3**	**65.4**	**54.7**

Table 11. **Comparison with concurrent work γ-MoD. All models are of 7B parameter scale.**

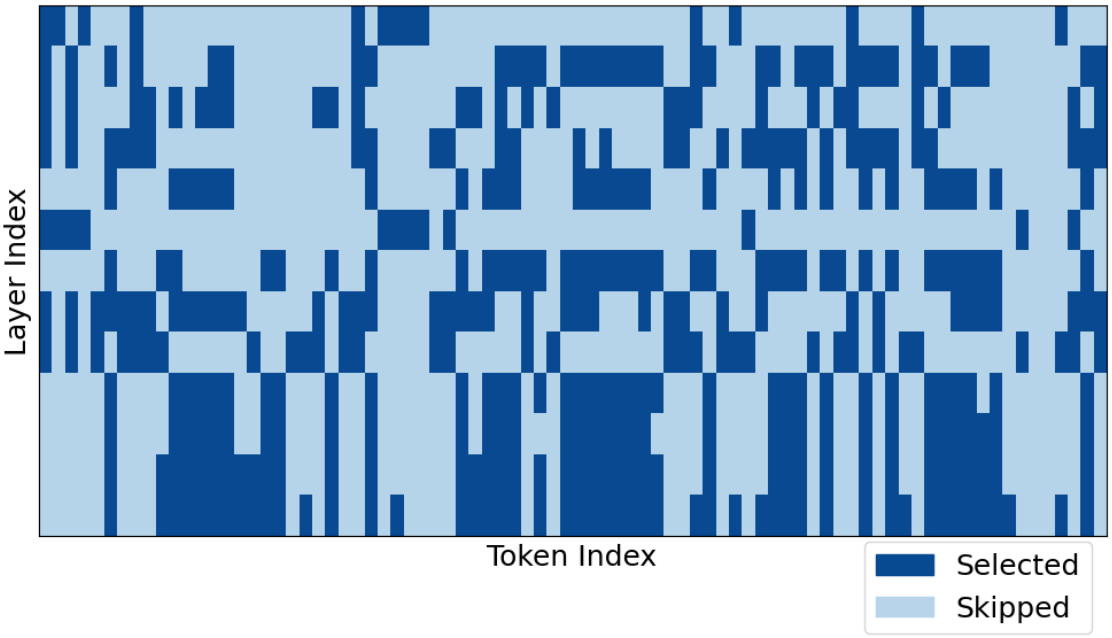


Figure 6. **Visualization of token selection decisions across different *p-MoD* layers.** The horizontal axis denotes the token indexes, and the vertical axis denotes the layer indexes. It can be observed that every ***p-MoD*** layer independently selects important and informative tokens.

B. Visualization of Token Selection Decisions

Figure 6 visualizes the token selection decisions of ***p-MoD*** across different layers. The horizontal axis denotes the token indexes, and the vertical axis denotes the layer indexes. It can be observed that every layer select different tokens to process, and every token is selected by different ***p-MoD*** layers. This demonstrates that every ***p-MoD*** layer independently selects important and informative tokens, without degrading into selecting a same set of tokens across different layers, which is identical to dropping tokens instead of layer-wise selection.

C. Further Discussions

C.1. Discussion on OCR Performance

In our experiments, we found that *all* token compression methods inevitably cause a significant performance drop on OCR-related benchmarks compared to other benchmarks. The main challenge we aim to address throughout the progress of this work is to mitigate this issue as much as possible. Experiment results in this paper show that our method achieves the *least* performance drop on OCR tasks compared to others.

C.2. Explanation on Training Overflow Problem

In Section 3.2.1, we mention that **TanhNorm** ensures training and inference stability. Acoordingly, we observe overflow issues in the ablation study on **TanhNorm** in Table 3. In this Section, we give a detailed explanation on the causes of overflow.

As illustrated in Equation 2, scaling a token by excessively large weights w_i across multiple LLM layers may cause **floating-point overflow**. For TanhNorm with $\alpha = 1$ (Table 3 Row 4), large w_i makes $X_i' = \alpha \tanh(w_i) T(X_i) + X_i$ approximate scaling the token X_i by 2. Repeating this for the same token across multiple layers causes overflow. The same problem arises for vanilla MoD (Table 3 Row 1). In this case, **no normalization** is applied to constrain the range of the weights, making it easier to produce extreme values and result in numerical overflow.

Name	Resolution	Train Stage	Trainable Module& Learning Rate	Data	Batch Size
p-MoD-LLaVA-v1.5	(336,336)	PT	Connector: 1e-3	558K	256
		SFT	Connector+LLM+MoD: 2e-5	665K	128
p-MoD-LLaVA-NeXT	336 × [(2,2), (1,2), (2,1), (1,3), (3,1)]	PT	Connector: 1e-3	558K	256
		SFT	ViT: 2e-6 (baseline) Connector+LLM+MoD: 2e-5	779K	128

Table 12. **Detailed training configuration**. PT stands for pre-training. SFT stands for supervised fine-tuning. During fine-tuning, the vision encoder of the baseline LLaVA-NeXT model is updated, consistent with the original LLaVA-NeXT model[28]. We freeze the vision encoder of our ***p-MoD***-LLaVA-NeXT model for all experiments to reduce training cost.

C.3. Limitations

One limitation of our work is that ***p-MoD*** is only experimented on LLaVA-1.5 and LLaVA-NeXT models, which focus on single-image understanding tasks. We believe that our approach has the potential to achieve more remarkable results when applied on tasks that handle a larger number of vision tokens, such as multi-image and long video understanding. We leave the exploration of ***p-MoD*** on other vision tasks to future research.

Another limitation is that ***p-MoD*** is a *trainable* method, designed for training a new MLLM from scratch with reduced training and inference costs. When being applied to trained MLLMs, it requires continual fine-tuning the model. It is not suitable for training-free scenarios.

D. More Implementation Details

D.1. Detailed Training Recipe

Our detailed training recipe of ***p-MoD*** models is shown in Table 12. LLaVA-1.5 employs a fixed input resolution of 336×336, while LLaVA-NeXT supports a pre-defined set of different resolutions (up to 672×672). Both the LLaVA-v1.5 models and the LLaVA-NeXT models go through the same pre-training stage, where the MLP connector module is trained on 558K image caption data [29] with a learning rate of 1e-3 and a batch size of 256.

During supervised fine-tuning, LLaVA-1.5 is trained on 665K instruction-tuning data [27], while LLaVA-NeXT is trained on a larger set of 779K data[*]. The vision encoder of the LLaVA-NeXT baseline model is updated for fine-tuning. For our ***p-MoD***-LLaVA-NeXT model, we freeze the vision encoder to save training time, as the available GPU resources are limited. When measuring the training GPU hours reported in Table 7, we freeze the vision encoder for both baseline and ***p-MoD*** models to ensure fair comparison.

[*]https://huggingface.co/datasets/lmms-lab/LLaVA-NeXT-Data

D.2. Hardware and Hyperparameters

We train all our models on 8 NVIDIA RTX A6000 GPUs. The inference efficiency metrics reported in Section 4.5 are measured on a single A6000 GPU. By default, we set the gating factor α in **TanhNorm** to 0.2, and the shift factor β in **PRD** to 0.5.

Due to computational constraints, we mainly use 7B models in our experiments. Experiments on 13B models are show in Table 8 and Section A.1.

D.3. Details on the PRD schedule.

We find it challenging for MoD layers to learn to predict meaningful weights when the token retention ratio is set to an extremely large or small value. To address this issue, we constrain the range of the token retention ratio R_l within a predefined maximum and minimum threshold. If R_l exceeds the maximum threshold, we set the token retention ratio to 1 so that the MoD module is not applied to the l-th layer. If R_l falls below the minimum threshold, the token retention ratio for the l-th layer is set to the minimum threshold:

$$R'_l = \begin{cases} 1, & \text{if} \quad R_l \geq max \\ R_l, & \text{if} \quad min < R_l < max \\ min, & \text{if} \quad R_l \leq min \end{cases} . \tag{6}$$